NOMILA-V2 · PHASE A · CURRENT → TARGET · 10 SEP 2026

Phase A Migration Map

Identity → Research → Canonical Athlete Profile → Social Intelligence → Marketability → Rates → Commercial Intelligence. For each module: what the Beta.1 specification asks for, what the codebase does today, and a KEEP / MODIFY / EXTRACT / REPLACE / ADD disposition for every component. No code has been written. Nothing here changes a LOCKED decision; where current code conflicts with one, it is raised in section 9.

target: Nomila Product & Intelligence Architecture Blueprint V1 / Commercial Intelligence Beta.1 (one docx, v3)

current: score-engine audit + recommendation-system map (local main 49b236d)

KEEP unchanged, already matches

MODIFY same component, changed contract or behaviour

EXTRACT pull existing logic out into the target's boundary

REPLACE retire and substitute

ADD does not exist

WHERE THE CODE ALREADY AGREES

Storage discipline, versioned weights, tap-to-act, source precedence

Append-only snapshots, a one-active-row weights table, the athlete-asserted ledger with provenance, a connected > discovery > research resolver, and pure engines that never query the database are all in place. These are KEEP.

WHERE IT DIVERGES MOST

The signal layer does not exist

Today raw platform aggregates go straight into four capitals through hand-set point tables. Beta.1's core is a Signal Engine with cohort percentiles, priors, confidence and provenance between evidence and score. That layer is ADD, and it is what the doc names the immediate milestone.

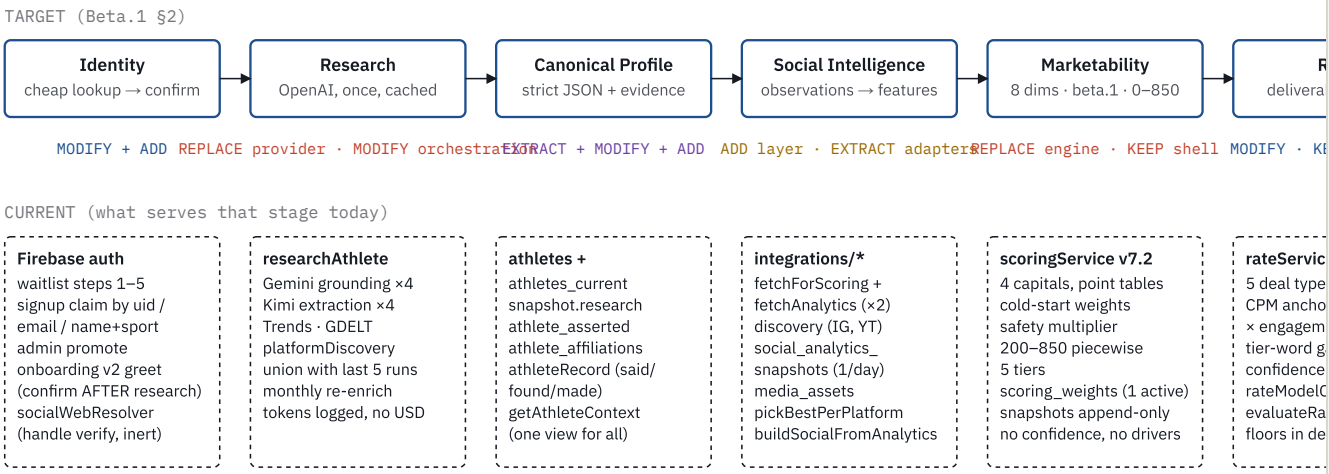
WHAT TO RAISE BEFORE TOUCHING ANYTHING

Six conflicts with LOCKED or frozen decisions

Research provider, the 0 vs 200 floor of the 850 scale, eight dimensions against four locked ring colours, tier vocabulary, commercial-proof

circularity, and research "never forgets" versus as-of dates. Section 9.

1 · The Phase A chain, target over current



Every target stage has something serving it today except Commercial Intelligence. The two stages with the largest gap are Social Intelligence (no observation or feature layer) and Marketability (a different model, not a different implementation of the same one).

2 · Identity

Target (§4): auth → confirm name → ask sport → cheap identity lookup → show the likely profile → athlete confirms → full research once → save canonical profile. Identity ambiguity is returned explicitly, never forced. **Current:** research starts the moment a sport is known (landing step 3, app signup, promote); the athlete reacts to what research found afterwards in the onboarding greet step; there is no "is this you" moment and no identityConfidence anywhere.

COMPONENT	DISPOSITION	WHY	FILES
Firebase auth, signup idempotency, claim-by-email,	KEEP	Matches "Google / Apple / Email Auth → Name + Sport". The claim rule is	server/routes/auth.ts · lib/auth.ts

claim-by-name-and-sport of pre-seeded rows		already a form of identity resolution against pre-seeded rows.	
Landing waitlist steps (name, school, sport, career status, goals, typed handles)	KEEP	Collects exactly the inputs the target lookup needs; handles typed here already override guessed ones in research.	`server/routes/waitlist.ts · landing/index.html`
Onboarding workflow v2: research → context_ready → greet → doors → wrap	MODIFY	Insert an `identify` step (cheap lookup, model or search, returns candidates + identityConfidence) and a `confirm` await_human step BEFORE research. The runner supports await_human already. Research becomes conditional on confirmation. Version bump to onboarding v3; live v2 runs finish on v2.	`server/workflow/definitions/onboarding.ts · server/mila/onboarding.ts (greet/doors parsers)`
startRun on sport-known in waitlist, signup, promote	KEEP	Trigger stays; what the run does first changes.	`waitlist.ts · auth.ts · admin.ts`
socialWebResolver (Claude web_search verifies a handle belongs to this athlete)	EXTRACT	This is the one piece of identity verification logic that exists. It belongs in the identity lookup step, not buried under platform discovery. Provider question raised in §9.	`server/lib/socialWebResolver.ts`
identityConfidence, candidate list, "withheld" state	ADD	No field or state today; athletes.status has	`athletes.status CHECK list (deploy.yml mirrors it)`

(Mia Carona precedent in §21.3)	no unresolved value.	

3 · Research

Target (§4–5): OpenAI as the generalized web-research provider behind a pluggable ResearchProvider interface; run once after confirmation; cached and versioned dossier; per-family refresh by TTL and events; strict versioned JSON; every claim with provenance, timestamp, source type and confidence; cost telemetry per job (model, tokens incl. cached, search calls, latency, USD). **Current:** four Gemini-grounded searches and four Kimi extractions per run, union-merged with the last five runs ("research never forgets"), plus Google Trends, GDELT and platform discovery; monthly full re-run for every active athlete; token counts logged, no USD, no search-call count; confidence only on brand mentions; identity handled by a search string.

COMPONENT	DISPOSITION	WHY	FILES
researchAthlete orchestration (queries → extract → merge → discovery → snapshot → score → current)	MODIFY	Keep the skill as the orchestrator; swap the provider calls for a ResearchProvider; stop calling computeScore inline once the signal engine owns scoring (the writer chain stays). Add asOfDate and cost telemetry to the run.	`server/skills/researchAthlet`
Gemini grounding + Kimi extraction as the research model layer	REPLACE	§4 names OpenAI as the V1 provider and says not to add Google grounding by default. Raised in §9 because it moves spend off Vertex credits and changes output shape. Do behind the interface so the swap is a config change.	`researchAthlete.ts runGemini/extractFromText · lib/models.ts · lib/kimi.ts`
ResearchProvider interface + OpenAI adapter + model router config	ADD	No abstraction exists; no OpenAI client exists (the openai package is used only	`new: server/lib/research/provider openaiProvider.ts`

		as Kimi's client library).	
Google Trends, GDELT count and tone	KEEP	Native public sources that feed Cultural Relevance (search interest, media volume) and Brand Safety; they become evidence rows with sourceClass platform_public rather than bare snapshot columns.	`server/lib/trends.ts · gdelt`
Union merge across the last five runs (researchMerge.unionResearch)	MODIFY	Conflicts with §21.1 as-of dates and signal-family freshness: an old achievement kept forever must not read as current momentum. Keep the dedup, stamp every item with observedAt and keep the "current" view a function of TTL. Raised in §9.	`server/skills/researchMerge.`
Monthly re-enrich of every active athlete	MODIFY	§4 cost rules: refresh signal families by TTL and events, not a full job per athlete per month.	`server/skills/reEnrichAthlet · index.ts cron 0 5 1 * *`
Kimi extraction schema (achievements with level, press, brand_deals with confidence, teams, story_arc, sentiment, media_assets)	MODIFY	Becomes the AthleteResearchProfile schema: schemaVersion, sections (athleticAuthority, audience, media, commercial, narrative), evidence[] with provenance and confidence, researchQuality, researchSummary, researchLimitations. Bounded taxonomies for AI-assisted classifications (§6).	`researchAthlete.ts EXTRACTION_SCHEMA`
Snapshot storage of research (append-only, per trigger)	KEEP	Matches "never overwrite history". Add	`athlete_snapshots.research · sources`

		dossier version and asOfDate columns.	
Cost telemetry (kimi_usage, gemini_usage, gemini_stats in snapshot.sources)	MODIFY	Partial today. Add search calls, cached tokens, latency, estimated USD, per research job; Langfuse tracing already on.	`researchAthlete.ts sources · lib/tracing.ts`
Found-team proposals from research (athlete_affiliations, source research)	KEEP	Already a provenance-carrying, athlete-confirmable fact.	`server/services/affiliations proposeFoundAffiliations`
Brand research (researchBrand, brandIntel)	KEEP for Phase A	Out of the Phase A chain; same provider swap applies later.	`server/skills/researchBrand.`

---

4 · Canonical Athlete Intelligence Profile and Context Builder

Target (§5): one canonical structured record with provenance per claim, versioned; an Agent Context Builder that produces purpose-specific views (Mila, Content, Deal, Commercial). **Current:** the profile is spread across athletes (identity, goals), athletes_current (score, rates, counts, story_arc), the latest snapshot's research, athlete_asserted (what she said, with source), athlete_affiliations (teams with kind and dates), and deal_preferences. getAthleteContext builds one view for the deck, Studio and shot list; the Narrator, chat v1, onboarding and Facts each build their own.

COMPONENT	DISPOSITION	WHY	FILES
athlete_asserted ledger (field, value, source, at) + profile_write receipts	KEEP	Already the "said" half with provenance. Its values outrank research in the canonical profile.	`shared/schema.ts athleteAsser · server/mila/onboarding.ts recordAssertions`
athlete_affiliations + levelRecordFor projection	KEEP	Competition level and career stage inputs for Athletic Authority and cohort selection.	`server/services/affiliations.`

athleteRecord said / found / made read with provenance	EXTRACT	This is the closest thing to a canonical read today. Its merge rules (said over found, made is computed) become the profile assembler.	`server/services/athleteRecord`
getAthleteContext (single deck-shaped view)	EXTRACT	Becomes the Context Builder with named views; the deck view keeps its current shape so mediaKitService, shotList and Studio brief do not change in Phase A.	`server/services/athleteContex`
buildNarratorContext, deriveShotList fallback query, chat.ts v1 inline read, loadOnboardingFacts	MODIFY	Point each at a Context Builder view instead of its own select, one at a time, after the builder exists. chat.ts v1 path can simply stay dormant.	`mediaKitService.ts` · `shotList` · `routes/chat.ts` · `mila/onboarding.ts`
AthleteResearchProfile as a versioned stored object; evidence table with provenance, timestamp, sourceClass, confidence	ADD	No per-claim provenance today except brand_mentions.confidence and the asserted ledger. §15 requires research, features, scores and rates persisted separately.	`new table(s); or snapshot.research under a schemaVersion with an evidenc index`
athletes_current story_arc, achievements_count, press_count, brand_mentions_count	KEEP as projections	Display conveniences; stop treating them as inputs (story_arc bonus is already 0).	`athletesCurrentWriter.ts`
athletes_current.insights, brand_stories; onboarding_sessions	REPLACE (drop later)	Unused; the canonical profile supersedes them.	`shared/schema.ts`
Goals, secondary goals, brand voice, deal preferences	KEEP	Athlete-declared context for the Deal and Content views; not Phase A scoring inputs.	`shared/goals.ts` · `routes/profile.ts`

5 · Social Intelligence (§22, §26, §28)

Target: the layered data model of §28.3 (identities, sync runs, sanitized source payloads, durable content, metric observations, full audience distributions, versioned features), platform adapters that preserve native fields, robust distributional features by content type, named metric definitions, the authority rule connected_verified > platform_public > web_researched > imputed, unknown ≠ zero, and scoring that imports no provider types. This is the doc's stated immediate milestone (§25, §31). **Current:** one flat row per athlete × platform × day with averages and top-N demographic columns plus a raw_metrics blob that copies recent media daily; media_assets as durable content with per-post insights; two fetch paths per platform (fetchForScoring for the score, fetchAnalytics for display); discovery for Instagram (10 to 25 media) and YouTube (10 videos); a best-row-per-platform resolver with a 45-day freshness window.

COMPONENT	DISPOSITION	WHY	FILES
OAuth providers, token refresh, connection lifecycle, expired-on-401 rule	KEEP	Unaffected by the data model.	server/lib/int · instagram.ts youtube.ts · oauthCallbac
Platform fetch code (Graph media + insights + demographics; TikTok video/list; YouTube Data + Analytics)	EXTRACT	The HTTP calls are right; what they return should land as raw payload + canonical observations, not as a pre-averaged SocialAnalyticsResult. Split each module into fetch (unchanged) and adapter (new).	instagram.ts tiktok.ts · yo discovery.ts
fetchForScoring per platform (the second, scoring-only fetch)	REPLACE	§28.2: scoring consumes versioned features, not a fetch. Once features exist, the connect-time path writes observations and the feature calculator runs; fetchSocialData stops calling computeScore directly.	instagram.ts:2 youtube.ts:128 skills/fetchSo
SocialAnalyticsResult / SocialAnalyticsRaw contract	MODIFY	Keep as the display contract for the Growth tab and get_analytics; it must stop being a scoring input (§28.2 "UI analytics contracts must not become scoring inputs").	integrations/t shared/schema.

social_analytics_snapshots (flat daily row + raw_metrics with recent_media copied daily)	KEEP during transition, then MODIFY	Stays as the display layer and the backfill source. §28.8 says stop copying full media payloads per day: after the observation tables exist, the daily row keeps account-level fields and raw_metrics stops carrying recent_media.	shared/schema. skills/collect
media_assets (durable content, insights jsonb, GCS rehosting, enrichTopPostInsights, mediaBackfill)	MODIFY	Already the "store durable content once" entity of §28.3 social_content. Add platform-native fields (TikTok duration, title and description separately; IG media_product_type; carousel children already kept) and stop forcing TikTok/YouTube into the Instagram shape. Per-post metrics move to observations with observedAt rather than overwriting insights.	shared/schema. skills/mediaBa enrichTopPostI
social_identities, social_sync_runs, raw_source_payloads, social_metric_observations, social_audience_distributions, social_features	ADD	None exist. connected_accounts holds the current handle and id but overwrites; the raw payload is not a wire log; demographics arrays live only inside raw_metrics; no feature table.	new migrations 0034 is taken #309)
Secret sanitization before persisting payloads	ADD and VERIFY	Graph paging URLs embed access_token; media_backfill_state stores a cursor. Confirm nothing token-bearing is stored today before the raw table is introduced.	instagram.ts (media_backfill
Engagement rate definitions (IG public: likes+comments per follower; IG connected: Σinteractions/Σreach; TikTok: per views; YouTube: per subscribers)	MODIFY	§28.6: name them precisely and never compare across definitions. Today buildSocialFromAnalytics puts them all in one `engagement_rate` field	integrations/* refreshScores. buildSocialFro scoringService platformEngage

		and the score compares IG to TikTok with a 0.75 factor.	
is_discovery heuristic (not on grid AND reach ≥ 2× followers)	MODIFY	§28.7: rename to a derived, versioned feature (broad non-feed distribution), never presented as provider truth.	`instagram.ts:4` `raw discovery`
Source precedence resolver (connected 4 > provider 3 > discovery 2 > research 1, fresh within 45 days)	EXTRACT	Already the §28.9 authority rule in spirit. Move it to the feature layer, apply per metric and window, and keep the superseded observation for audit.	`refreshScores.` `pickBestPerPla`
Discovery coverage: IG Business Discovery 10 to 25 media; YouTube 10 videos; TikTok public none; DISCOVERY_IG token config	MODIFY	§26.2 and §28.5: paginate YouTube to a defined window, request the additional IG media metadata Meta exposes, keep TikTok public as unknown (imputed), fix Business Discovery permission failures rather than replacing the layer.	`discovery.ts` `· youtube.ts`
TikTok demographics behind DEMOGRAPHICS_ENABLED / creator_info	KEEP dormant	§28.5 says do not depend on it until the approved product is verified.	`tiktok.ts:23-3`
Instagram Stories analytics	ADD	Not collected; §28.5 asks for connected Story analytics with a capture cadence suited to ephemerality.	`instagram.ts`
Full demographic distributions retained in raw_metrics.demographics; top_* columns	KEEP → migrate into social_audience_distributions	Already preserved (§26.4). The top_* columns stay as display fields.	`shared/schema.` `SocialAnalytic`
Versioned feature calculator (median, p25/p75/p90, per-1,000 rates, efficiency, breakout rate, CV, cadence, segmentation)	ADD	Only means exist today. The calculator must run from stored observations without refetching.	`new:` `server/service`
Mia fixtures from the 9 Sep export; the social data	KEEP	§27, §30 step 2, §31 tests. docs/social-data-	`docs/social-da`

inventory already written		inventory.html (untracked) is the §23.1 field export; it should become the "fields intentionally not collected" record too.	
Nightly collect → discovery → refresh cron	MODIFY	Keep the schedule; the refresh step recomputes features and then the engines; add the advisory lock the score audit flagged.	server/index.t

---

6 · Marketability (Beta.1, frozen)

Target (§6–§9, §21): eight dimensions with frozen top-level weights 15/15/15/10/10/10/15/10 and the sub-signal tables of §7; every sub-signal a NormalizedSignal (0–100, confidence, source class, evidenceIds, modelVersion) produced by the Signal Engine through cohort percentiles, versioned lookup tables or bounded AI classification; priors hierarchy and separate Data Confidence; composite 0–100 mapped by a versioned calibration function (alpha: 850 × (c/100)^0.8); signal ownership, as-of dates, freshness per family, career status separate from commercial activity; drivers and an explanation graph; the 37-athlete cohort as a regression baseline; no athlete-specific patches. **Current:** v7.2 with four capitals built from hand-set point tables, follower log curve without cohort normalization, achievement keyword lists, conference lists, cold-start weight profiles, a GDELT-tone safety multiplier, a piecewise 200–850 transform, five tier words, no confidence, no drivers, and a momentum block nobody reads. Weights are versioned in scoring_weights; snapshots are append-only; the engine is pure; the writer is single.

COMPONENT	DISPOSITION	WHY	FILES
computeScore v7.2 internals (capitals, keyword lists, cold-start profiles, safety multiplier, display transform)	REPLACE	A different model, not a re-implementation. Build the Beta.1 engine beside it, select by weights-row version, shadow-run, cut over. The score-engine audit's transition plan applies.	server/services/scoringService.ts

Pure-engine boundary (engine takes a snapshot and returns a result; never queries)	KEEP	Matches §15 reproducibility. The new engine takes a feature vector instead of a raw snapshot.	`scoringService.ts contract · snapshotAdapter.ts`
scoring_weights one-active-row table + reloadWeights	MODIFY	Becomes the versioned parameter store for weights, sub-weights, lookup tables, priors and the calibration exponent. Drop the dead rateEstimation requirement first (crashes an otherwise clean row).	`weightsService.ts · migrations 0038, 00 pattern`
Snapshot per scoring run with the whole result JSON; athletes_current projection; the single writer	KEEP	"Never overwrite history" and "persist separately" are satisfied. Add a features snapshot beside it.	`athlete_snapshots.score · athletesCurrentWriter.ts`
Signal Engine: feature extraction → cohort selection → percentile / lookup → NormalizedSignal	ADD	Nothing today normalizes against a cohort; there are no cohort tables and no priors. This is the defensibility layer the doc describes.	`new: server/services/marketability/signals.t cohorts, priors`
Cohort benchmark tables (sport × level × stage × platform) and the 37-athlete + holdout cohorts	ADD	golden_athletes holds 5 rows with PAV ground truth; no marketability targets, no cohort. The benchmark harness (compare, error, tier match, rank correlation) is the freeze gate of §20.	`extend golden_athletes or new benchmark table · new script like scripts/_dryRunV72.ts`
Athletic Authority inputs: competition level from affiliations, achievements with level from research	MODIFY	Inputs exist; the classification of achievement level moves from in-code keyword lists to research-provided bounded taxonomy plus versioned	`scoringService.ts NATIONAL_KW/CONFERENCE_KW · affiliations.levelRecordFor`

		lookup tables (also resolves the no-hardcoding rule).	
Commercial Proof inputs (snapshotDealsFor: accepted/completed deals, total value, settled flag)	MODIFY	§14: only validated, capped commercial-proof evidence may influence Marketability. Today "accepted" is set by the summarizer's own read of a thread. Raised in §9.	`server/services/snapshotDeals.ts · threadSummarizer.ts advanceDealStage`
Momentum block in the result	EXTRACT	Its deltas (followers, engagement, trends, GDELT, new deals) are the raw material for the Momentum dimension; the calculation moves into the Signal Engine and gets a reader for the first time.	`scoringService.ts computeMomentumData`
Brand Safety: GDELT tone multiplier and sentiment fallback	EXTRACT	Becomes the brand-safety sub-signal inside Brand Fit (20% of that dimension) rather than a global multiplier.	`scoringService.ts computeBrandSafetyMultiplier`
Display transform (200–850 piecewise) and tier thresholds	REPLACE	Alpha mapping is 850 × (composite/100)^0.8 on a 0–850 scale. Floor and tiers are conflicts to raise (§9).	`scoringService.ts computeDisplayScore · tierThresholds`
Data Confidence, drivers, explanation graph, "why did my score change"	ADD	Nothing today. scoreHistoryRoute reports deltas only. The MarketabilityResult interface in §17 carries confidence, dimensions, drivers, version.	`new · voice/scoreHistoryRoute.ts reads snapshot JSON`
refreshScores unchanged-	MODIFY	Compare the persisted result	`skills/refreshScores.ts unchanged()`

comparison (score, tier, capitals, rate card)		shape of the new engine; features moving daily will otherwise write every day.	
Readers of display_score, tier, capitals (app rings, Mila widget, voice card, deck index card, deal prompt, history)	KEEP via adapter, then MODIFY	Score-engine audit §F: keep a projection to the current shape during shadow and cutover. Eight dimensions against four rings is a product decision, raised in §9.	see score-engine audit, section F
Mila prompt line "Deals do NOT affect the M.I.A. score"	MODIFY	Commercial Proof is 15% of the target; the line must change to the validated-and-capped rule.	server/mila/prompt.ts:86

---

7 · Rates

Target (§11, §17, §22.4): deliverable taxonomy across social, brand content, in-person, long-term and rights add-ons; base_rate = benchmark(deliverable, sport, level, audience band) × audience × engagement × recognition × marketability × sport commerciality × demand momentum × commercial proof, plus usage, paid media, whitelisting, exclusivity, production, travel; RateEstimate with low/expected/high, confidence, assumptions, drivers, version; verified distributions as direct pricing inputs; calibrated independently of Marketability against the 37 rate targets. **Current:** v3.1 prices five deal types plus PAV from CPM anchors × athlete premium × engagement adjustment × a score modifier built from the capitals, with tier-word gates for appearance and autograph, provenance-driven confidence bands, constants in a TypeScript module, a backtest against IMH bands and three published estimates, and floors set by the athlete.

COMPONENT	DISPOSITION	WHY	FILES
Rate as a separate engine from the score; ranges not points; confidence by provenance; computed in the writer from the score result	KEEP	All three are LOCKED principles and already true.	rateService.ts · athletesCurrentWriter.ts

Formula shape (benchmark × factors) and the per-platform social base with log-log interpolation	MODIFY	Same skeleton as the target; add the missing factors (recognition, sport commerciality, demand momentum, commercial proof), replace the scoreMod capital blend with the marketability factor from the new engine, and take verified distribution features (median Reel views, p75) as direct inputs.	`rateService.ts computeRate( scoreMod`
Deliverable taxonomy (5 types today vs ~20 in §11) and rights add-ons (FORMAT_MULT, COMMERCIAL_ADDON exist but are unused in the card)	ADD	Extend the taxonomy; make add-ons first-class RateEstimate components rather than constants nothing reads.	`rateModelConfig.ts`
rateModelConfig constants	REPLACE with a versioned parameter table	§15 versioning; the module header itself says Phase 2 moves it to a draft→active table like scoring_weights.	`rateModelConfig.ts`
Tier-word gates (prestigeTier, autographTier) and the missing-capital default of 50	REPLACE	Silent failure modes flagged in the score audit; the target gates on audience band, level and marketability factor with explicit assumptions.	`rateService.ts:75-100`
RateEstimate persistence with assumptions, drivers, version; rate snapshots per run	ADD	rate_card is overwritten on athletes_current; only the score snapshot keeps history. §13 wants rate history and explanations.	`new table or snapshot colu`
rate_card, deal_type_rates, pav projections and their readers (deal	KEEP as projections	Every reader goes through deriveDealTypeRates	`rateService.deriveDealTypeI dealVoice.resolveAthleteRa deckRates.ts`

scorer, drafts, counters, kit, Mila, voice)	during transition	or resolveAthleteRate; keep those two functions as the adapter from RateEstimate[] to the current shapes.	
Athlete floors (deal_preferences.minimum_rates) and withFloors	KEEP	Athlete-declared; unchanged by the engine.	`routes/deals.ts PATCH /pre: · dealDetector.withFloors`
evaluateRates backtest, golden_athletes with ground_truth_pav, seedGoldenAnchors	EXTRACT	Seed of the Rate calibration harness; add the 37 rate targets and per-deliverable comparison.	`server/scripts/evaluateRat seedGoldenAnchors.ts`
Deal-side re-bake after a rate change (_rescoreImpacted)	KEEP	Required step at cutover; already exists.	`server/scripts/_rescoreImp`

8 · Commercial Intelligence: Opportunity, Earnings, outcomes, learning

Target (§12, §14, §15): an Opportunity Engine (demand, fit, conversion, capacity, seasonality, repeat), an Earnings Engine (conservative / expected / upside 12-month, plus Optimized Nomila Potential), capture of observed outcomes (offer vs suggested vs closed price, deliverables, rights, duration, performance, renewal, satisfaction, funnel), and a Learning Engine that calibrates Rate and Opportunity first and only later, capped, Marketability. **Current:** nothing computes opportunity volume or earnings; PAV_VOLUME_MIX is a hard-coded units-per-year table by bucket; deals, deal_periods (0 rows), payments and pending_actions hold outcome facts but not "suggested price at the time"; no learning code; offline evals for the classifier and Mila; Langfuse tracing on.

COMPONENT	DISPOSITION	WHY	FILES
PAV_VOLUME_MIX and pavBucket	EXTRACT	The seed of the Opportunity Engine's volume scenarios; becomes versioned parameters with	`rateModelConfig.ts · rateService.ts pav`

		conservative / expected / upside volumes.	
Opportunity Engine, Earnings Engine, EarningsPotential persistence	ADD	Not built.	new
Outcome capture: suggested rate at offer time, negotiated and closed price, rights, duration, renewal, funnel stage, satisfaction	ADD (partly MODIFY)	deals carries amount, cash/product split, commission, term dates, source, direction; inbox_threads.score carries the suggested rate only inside prose; pending_actions and agent_actions give funnel events; payments give settlement. Missing: a per-deal record of Nomila's suggested price and version at the time, renewal linkage, satisfaction.	shared/schema.ts deals · payments · pending_actions
Deal economics reads (dealValueStore, dealScoreGrid economics bands)	KEEP	Consumers of rates; unchanged in Phase A beyond the projection adapter.	services/dealValueStore.ts · skills/dealScoreGrid.ts
Learning Engine with versioned proposals and validation protocol	ADD later	§30 step 7: only after multiple connected cases and outcomes. Phase A only lays the outcome schema.	new
Classifier eval fixtures, Mila golden eval, Langfuse	KEEP	Existing validation discipline; the benchmark harnesses of §6 and §7 sit beside them.	server/scripts/evalClassifier.ts · scripts/run-mila-eval.ts

9 · Conflicts with LOCKED or frozen decisions, raised before any change

Each of these is either a LOCKED principle (§19, §28.1) that current code violates, or a place where the target's own choice cuts against something the product has already fixed. None is

changed in this map; each needs your call.

#	DECISION AT STAKE	CURRENT CODE	WHAT HAS TO BE DECIDED
1	§4 research provider: OpenAI as V1 web-research provider; Google grounding and Tavily not by default	Gemini grounding on Vertex (GCP credits, models.ts) + Kimi extraction; a Claude web_search handle resolver; Gemini retirement already planned for October	Confirm OpenAI for research and whether Vertex spend or credits matter. The interface makes it swappable either way; the default is your call.
2	§9 scale 0–850 with MIA = 850 × (c/100)^0.8	Display floor is 200 (segment1MinDisplay); every UI says "of 850"; tiers Icon 800 / Elite 700 / Established 550 / Rising 400 / Local 200	Whether the floor moves to 0 (a new athlete could read 0) and what the tier vocabulary and thresholds are. MarketabilityTier is undeclared in the spec.
3	§7 eight dimensions	Four capitals with LOCKED ring colours mapped to the logo (ring score design constraint memory); rings, Mila widget, voice card and deck index card all draw four	How eight dimensions present on a four-ring surface: group into four, add rings, or keep four "capitals" as a presentation roll-up of eight scored dimensions. This is product, not engineering.
4	§14 / §19 commercial outcomes calibrate without circular self-inflation; only validated, capped proof influences Marketability	Commercial capital counts deals whose "accepted" status was set by the summarizer reading a thread (inbox_scan source); confirmedPartnersFor already restricts the deck to contract_upload and self_reported	Which sources count as validated commercial proof for the score, and the cap.
5	§21.1 as-of dates and signal-family freshness	Research union-merges the richest list from the last five runs and never forgets	Keep dedup but stamp observedAt and expire by family TTL; confirm the TTLs are yours to set as alpha parameters.
6	§4 identity confirmation before full research	Research runs at signup; the greet step confirms afterwards; Mia's and Lilia's v2 runs are live	Onboarding v3 with identify + confirm before research. Existing runs finish on v2. Affects the landing page copy the founder already approved.
7	§28.1 no paid social-data provider; fix Business Discovery permissions rather than replace	Compliant: no Modash/HypeAuditor; DISCOVERY_IG_* configured; TikTok public absent	None. Noted so it is not reopened.

8	§19 connection is an accuracy event, not a bonus	Cold-start weight profiles change all four weights when social data appears; verified data does not carry higher confidence, it carries a data_tier label	Replaced by priors + confidence in the new engine; no decision needed, but the shadow run should show Mia's score can move either way.
9	§28.10 no scoring code imports provider types	scoringService imports none; refreshScores maps DB rows; fetchSocialData maps provider results directly into the scoring snapshot	Satisfied once fetchForScoring is retired (section 5).
10	Migration numbering and environment split	0034 exists on main (workflow runner) and in PR #309 (trend_observations); origin/main now ships one schema script for every environment (#375) and a promotion pipeline (#372)	New tables in Phase A must use the upstream schema-script flow; pull origin/main before the first migration.

---

10 · Execution sequence, for discussion

You asked to work module by module in the order Identity → Research → Profile → Social → Marketability → Rates → Commercial. The spec's own definition of done (§25, §30, §31) puts the Social Intelligence data layer first. Both orders end in the same place; the difference is what ships first without touching production behaviour. My recommendation is below, with the reason. Nothing starts until you pick.

0 **Foundations, no behaviour change.** Versioned parameter store (extend scoring_weights or a new table), asOfDate on snapshots, the NormalizedSignal / SocialMetricObservation / RateEstimate types, and pulling origin/main so migrations use the new schema flow. Small PR.

1 **Social Intelligence data layer (section 5).** Observation and feature tables, adapters extracted from the existing fetch code, secret sanitization, the feature calculator, Mia fixtures, and the one command that produces her feature vector for an asOfDate. Additive: the daily snapshot and the current score keep running. This is the spec's immediate milestone and the input every later engine needs.

2 **Research provider and canonical profile (sections 3 and 4).** ResearchProvider interface, OpenAI adapter run in shadow beside Gemini for the ten live athletes, the

AthleteResearchProfile schema with evidence and provenance, cost telemetry, TTL-based refresh. Decision 1 and 5 first.

3 **Identity confirmation (section 2).** Onboarding v3 with identify + confirm before research. Decision 6 first. Small once the provider exists.

4 **Marketability Beta.1 in shadow (section 6).** Signal Engine, cohort tables, priors, confidence, drivers, the 37-athlete + holdout harness, shadow scores for the live athletes, then cutover by weights-row swap with the projection adapter from the score audit. Decisions 2, 3, 4 first.

5 **Rate Engine v1 (section 7).** Taxonomy, factors, parameter table, RateEstimate persistence, calibration against the 37 rate targets independent of the score, re-bake of deal verdicts.

6 **Commercial Intelligence (section 8).** Outcome capture schema first, then Opportunity and Earnings engines, then the Context Builder views for the Commercial and Deal agents.

What I need from you to start module 1

- Confirm the order above, or that Identity goes first as you listed.
- Decisions 1, 2, 3 and 4 can wait until their module; decision 10 (pull origin/main, use the new schema flow) is needed before the first migration.
- Whether the four existing untracked docs in docs/ (two audits, the social inventory) should be committed; the docs/audits-sep2026 branch is pushed but has no PR, pending your earlier "no".